

Team and Project Plan

Portfolio Task 1

Unit code: COS40005

Unit name: Computing Technology Project A

Submission date: 26th of September, 2025

Table of Contents

Introduction	5
Part 1: Team Charter and Code of Conduct.....	7
1. Detailed Team Profile.....	7
2. Roles and Responsibilities	12
3. Teamwork Roadmap and Collaboration Framework.....	14
3.1. Meeting Cadence	14
3.2. Communication Plan.....	14
3.3. Task Management Workflow	15
4. Document and Configuration Management.....	17
4.1. Code Version Control.....	17
4.2. Document Management.....	17
4.3. Design and Architectural Asset Management	18
5. Risk Management.....	19
Part 2: Project Overview and Plan	22
6. Project Background and Problem Statement	22
6.1. Background	22
6.2. Problem Statement	22
7. Project Goals and SMART Objectives.....	23
7.1. Project Goal.....	23
7.2. SMART Objectives.....	23
8. Scope, Assumptions and Constraints.....	25
8.1. In-Scope	25
8.2. Out-of-Scope	25
8.3. Assumptions.....	25
8.4. Constraints	26
9. Stakeholders.....	27
10. Detailed Solution Approach and System Architecture.....	29
10.1. Preliminary Design of Patient-facing Dashboard	29
10.2. Preliminary Design of Clinician-facing Dashboard.....	36

10.3.	Frontend Architecture: A Cross-Platform Strategy	41
10.4.	Backend Architecture: An AI-Centric Python Core	41
10.5.	Data and BaaS Architecture: A Secure, Open-Source Foundation	42
10.6.	AI Core Architecture: An Agentic System	43
11.	Detailed Product Backlog	46
12.	Quality Assurance Plan	50
12.1.	Code Quality Standards.....	50
12.2.	Testing Strategy	50
12.3.	Defect Management.....	51

Table of Figures

Figure 1: Main dashboard (website)	29
Figure 2: Main dashboard (mobile application)	30
Figure 3: AI assistant chat interface (website).....	31
Figure 4: AI assistant chat interface (mobile application)	32
Figure 5: Patient's profile page (website)	33
Figure 6: Patient's profile page (mobile application)	34
Figure 7: Main dashboard	36
Figure 8: Sort function	36
Figure 9: Patient's information overview	37
Figure 10: Recent automated actions	37
Figure 11: Glucose and HbA1c trends, and physical activity.....	38
Figure 12: Diet log and automated actions log	38
Figure 13: Notes and communication	39
Figure 14: Notification centre	39
Figure 15: Clinician profile.....	40
Figure 16: Conceptual diagram of multi-layer technical architecture.....	44
Figure 17: System architecture	45

Introduction

This document serves as the comprehensive Team and Project Plan for developing the "AI-Enabled Digital Health Platform for Chronic Disease Monitoring," undertaken on behalf of BioTective Sdn Bhd. A formal charter aims to provide a single source of truth for all project stakeholders, outlining the team structure, communication protocols, project scope, technical architecture, development roadmap and quality assurance strategy.

The project's vision is to create a functional, high-fidelity prototype demonstrating the feasibility and value of leveraging advanced AI to transform chronic disease management. This involves moving beyond simple data logging to create an intelligent, proactive system that offers personalised insights and automates timely interventions. This plan is a living document, designed to guide the team from initial setup to the final delivery of a robust and polished prototype, laying the groundwork for a future production-ready system.

Student name	Student ID	Description of contribution in team and project planning
Daniel Tiong	102777801	Coordinated the overall project planning process, led the definition of the technical and AI architecture and drafted the primary sections of the project plan, including the detailed solution approach and risk mitigation register.
Alif Harriz	102782711	Contributed significantly to the front-end technology selection and UI/UX strategy. Provided critical input on user-centric design principles and accessibility considerations, and helped define the teamwork roadmap and communication plan.
Ashley Jong	102780087	Assisted in the detailed breakdown of project requirements from the initial proposal,

		contributing to the system analysis and creating a granular product backlog with user stories. Also helped structure the quality assurance plan.
Basil Agas	102778888	Provided key insights during the AI framework analysis, influencing the decision to choose the model and the framework. Contributed to defining the AI-related tasks and deliverables within the project plan.
Edison Ho	102779496	Led the in-depth analysis of backend, BaaS, and database technologies. Authored the rationale for selecting the platform and contributed to outlining the document management, security framework, and quality plan sections.

Table 1: Description of contribution in team and project planning

Part 1: Team Charter and Code of Conduct

This section defines the team's internal operating structure, roles, responsibilities and the agreed-upon protocols for communication, collaboration and conflict resolution. It forms the foundation of the team’s professional conduct.

1. Detailed Team Profile

The project team is a well-rounded, multidisciplinary group of computer science students, each bringing a unique specialisation that maps directly to the project's complex requirements. The synergy between their skills in Artificial Intelligence, Software Development, UI/UX Design and Full-Stack Engineering creates a robust unit capable of tackling this ambitious project.

Daniel Tiong	
Technical skills	Full-stack development, from front-end to back-end. AI/ML experience, including agentic systems and image classification. Skilled in creating multi-platform desktop applications, IoT solutions and small-scale games. Strong UI design skills with an emphasis on clean, minimalist and professional aesthetics. A tendency to focus heavily on perfection can sometimes slow down project timelines. A desire for high-quality work can make it challenging to delegate tasks, as it takes time to understand and integrate the work of others.
Soft skills	High attention to detail ("pixel perfect"). Driven by a passion for creating efficient, automated systems to streamline processes. Strong, user-focused design sense.
Communication	Proactive communicator, acting as the primary point of contact for the team with the client. Comfortable with formal written communication.
Teamwork	Takes initiative and acts as a project coordinator. Full-stack knowledge bridges the gap between front-end, back-end and AI components to ensure seamless integration.
Others	Focused on building complete, end-to-end, innovative, functional, and aesthetically pleasing solutions.

Table 2: Daniel's profile

Alif Harriz	
Technical skills	Front-end web technologies such as HTML, CSS and JavaScript. Experience with modern JavaScript frameworks, particularly Bootstrap and Vue.js. Familiarity with design and prototyping tools like Figma to create wireframes and interactive prototypes. Furthermore, capabilities extend to high-performance mobile and desktop development using Flutter. Leveraging the Dart programming language, building natively compiled mobile (iOS, Android), web, and desktop applications from a single codebase. This approach creates beautiful, fast, and expressive UIs with a consistent user experience across all platforms, significantly accelerating development and deployment timelines.
Soft skills	A sense of user empathy, focusing on creating intuitive and accessible user experiences for both tech-savvy and non-technical users.
Communication	Prefers to present ideas and solutions using prototypes to ensure a clear understanding with the team. Comfortable communicating via asynchronous chat (WhatsApp) for quick updates.
Teamwork	Focused on translating the agreed-upon designs and requirements into a functional, polished product during the design and planning phases. Open to any constructive feedback.
Others	Committed to applying accessibility best practices to ensure the platform is usable and stress-free for a diverse patient demographic, including older adults.

Table 3: Alif's profile

Ashley	
Technical skills	Experienced in software development and artificial intelligence, with skills in software design, system analysis, and AI-driven problem solving. Exposure includes AI/ML concepts such as search algorithms, data preprocessing, model training, and evaluation, supported by coursework and projects. Additional experience covers project management, requirement gathering, software testing, cloud computing architecture, and version control. Hands-on academic projects applied AI and software engineering practices to deliver practical, efficient, and user-focused solutions.
Soft skills	Detail-oriented and methodical, with a structured approach to problem-solving. Adaptable in learning new tools and technologies, with strong communication and organisational skills that support efficient coordination, requirement gathering, and project delivery. Committed to creating intuitive, user-focused solutions that are aligned with project goals.
Communication	Comfortable with both formal written communication and technical discussions. Adaptable to different channels such as email, WhatsApp, Teams, and Discord.
Teamwork	Responsible for bridging front-end design with back-end functionality, ensuring smooth integration and a cohesive user experience. Handles tasks reliably, works well with feedback, and focuses on building features that meet the project's design and technical requirements.
Others	Improvement is needed in writing more efficient code and managing time effectively when handling multiple tasks. There is also room to grow in breaking down complex problems into smaller, actionable steps. Building greater confidence in technical decision-making and presentations remains a key focus for development.

Table 4: Ashley's profile

Basil Agas	
Technical skills	Specialised in AI/ML model development and integrating AI-powered features using frameworks like TensorFlow, PyTorch, Langchain, and Matplotlib. Proficient in Python back-end development, creating APIs to connect AI models with front-end applications, and agile in adopting optimal technologies for practical problem-solving.
Soft skills	The key to any practical project is clear communication, strong analytical and problem-solving skills, and focusing on implementing practical solutions when faced with real-world problems. Translating complex technical concepts into clear goal paths ensures that all project members understand the objectives and can communicate them effectively.
Communication	Prefers structured communication, like detailed project updates or meaningful weekly team meetings, to discuss progress and resolve complex issues. We are adaptable and flexible to the client's preferred communication method, including options like email, WhatsApp, Teams or Discord.
Teamwork	Serves as the AI system integrator, responsible for the entire AI lifecycle, from data collection and model training to deployment and maintenance. Works with the front-end and back-end team to ensure the AI's functionality is seamlessly integrated into the user interface and overall product.
Others	Experienced in data analysis and generating and utilising insights from user data to inform project decisions.

Table 5: Basil's profile

Edison Ho	
Technical skills	Proficient in full-stack web development with hands-on experience building applications from concept to deployment, including a student management system and multiple dynamic websites. My technical expertise covers front-end technologies (HTML, CSS, JavaScript, Bootstrap) for creating responsive user interfaces and back-end languages (PHP, Python) for server-side logic and database management. I also possess a foundational knowledge of cybersecurity principles, allowing me to build more secure and robust solutions.
Soft skills	A developer dedicated to creating high-quality, user-centric products. I strive to write clean, maintainable code to ensure easy integration and efficient troubleshooting for the team. I would look through products to be cohesive and have an intuitive user interface (UI). I always consider the user's journey and interaction with the application to deliver a practical and satisfying user experience (UX).
Communication	A proactive and transparent communicator who believes in fostering alignment within a team. I am comfortable presenting ideas, prototypes, and technical concepts to ensure everyone is on the same page. I believe in voicing concerns constructively and early to prevent roadblocks. I am adaptable in my communication style and proficient in in-person collaboration through platforms like WhatsApp, GitHub, and Discord.
Teamwork	A reliable and efficient team player who takes ownership of assigned tasks and executes them promptly. I excel at coordinating with teammates whose work depends on mine, ensuring a smooth workflow and successful integration between different project parts. I focus on contributing effectively to the team's shared goals and ensuring all moving parts are aligned.
Others	My core professional goal is to create impactful products that deliver value and satisfaction to both the client and the end-user. I am driven by the challenge of balancing business requirements with a seamless user experience, ensuring that the final solution is technically sound, genuinely useful, and enjoyable to use.

Table 6: Edison's profile

2. Roles and Responsibilities

To ensure clear ownership and accountability, specific roles have been assigned. These roles are not strictly siloed; collaboration is expected and encouraged. However, the designated role owner is primarily responsible for the quality and timely completion of their domain's deliverables.

Role	Assigned to	Justification and core responsibilities
Project lead and AI engineer	Daniel Tiong	With a full-stack perspective and a major in AI, Daniel is ideally suited to lead the project, ensuring all parts (frontend, backend, AI) integrate seamlessly. Responsibilities: Facilitate team meetings, manage the project backlog in GitHub, act as the primary liaison with the project supervisor, resolve team blockers and co-develop the core AI framework.
Frontend developer (patient dashboard)	Alif Harriz	Alif's focused expertise in front-end development and UX design is perfect for creating the data-rich, intuitive, and responsive patient dashboard. Responsibilities: Translate designs into functional widgets, integrate the chatbot interface, ensure the dashboard is performant and visually polished, implement data visualisations for patient view, and champion accessibility standards.
Frontend developer (clinician dashboard)	Ashley Jong	Ashley's double major in Software Development and AI, combined with her user-focused approach, makes her well-suited to build the clinician-facing dashboard. Responsibilities: Develop the clinician dashboard UI, ensure patient data is displayed clearly and effectively, and collaborate on testing and requirement validation.
Lead AI engineer	Basil Agas	Basil's specialisation in the architectural and operational aspects of AI systems makes him the ideal candidate to

		lead the development of the core recommendation engine and automation triggers. Responsibilities: Implement the AI framework, develop the tools for the LLM agent, and optimise AI framework performance and response accuracy.
Backend developer	Edison Ho	Edison's hands-on experience building full-stack applications and managing databases is critical for the project's foundation. Responsibilities: Develop the Python backend API, design and implement the database schema, configure Row-Level Security policies, and ensure seamless data flow between the frontend and the database.

Table 7: Team roles and primary responsibilities

3. Teamwork Roadmap and Collaboration Framework

This framework outlines the processes and tools the team will use to maintain momentum, ensure transparency and foster a productive working environment.

3.1. Meeting Cadence

Class Sync (Twice Weekly)

Mandatory meetings will occur during scheduled class times on Tuesdays and Thursdays. These sessions will be used for formal progress reviews, sprint planning and supervisor consultations.

Working Sessions (Twice Weekly)

Additionally, less formal working sessions will be scheduled on Fridays and Saturdays by default unless the scheduled working sessions are requested to be amended to any other day for the week, subsequent week, or indefinitely. These are hands-on, collaborative sessions dedicated to coding, debugging and integration.

Mode of Operation

The default mode for all meetings will be in-person to facilitate high-bandwidth communication. An online option via Google Meet will be available for members who cannot attend in person.

3.2. Communication Plan

Informal/Daily Communication

WhatsApp will be the primary channel for quick questions, daily updates and urgent coordination. A maximum response time of 12 hours is expected.

Formal Communication

Email will be used for all official communication with the project supervisor and for documenting key decisions.

Technical Discussion

GitHub Issues and Pull Requests will be the venue for all technical discussions, code reviews and bug tracking to keep conversations linked to the relevant code.

3.3. Task Management Workflow

Tool

The team will exclusively use *GitHub Projects* and *GitHub Issues* for all task management.

Lifecycle of a Task

Each task will be created as an Issue and will progress through the following stages on the project board:

- **Backlog:** Awaiting prioritisation.
- **To Do:** Prioritised for the current sprint.
- **In Progress:** Actively being worked on by an assigned team member.
- **In Review:** A Pull Request has been opened and is awaiting code review.
- **Done:** The Pull Request has been approved and merged, and the feature meets the Definition of Done.

Definition of Done (DoD)

A task is considered "Done" only when it meets all the following criteria:

- Code is successfully merged into the main branch.
- Code is formatted and linted according to project standards.
- All related acceptance criteria are met.
- The feature has been manually tested and verified by someone other than the author.
- Relevant documentation (if any) has been updated.

Conflict Resolution

1. Disagreements should first be discussed directly and respectfully between the involved team members.
2. If a resolution cannot be reached, the issue will be brought to the next team meeting for a group discussion, mediated by the Project Lead.
3. If the team cannot reach a consensus, the Project Lead will make a final decision after considering all viewpoints.
4. For critical issues that impact the project scope or timeline, the matter will be escalated to the Project Supervisor for guidance.

4. Document and Configuration Management

A systematic approach to managing project assets is crucial for consistency, collaboration and maintaining a single source of truth.

4.1. Code Version Control

- **System:** Git
- **Platform:** GitHub
- **Branching strategy:** The team will adopt the **GitFlow** branching model.
 - o *main*: Contains production-ready, stable code. Direct commits are forbidden.
 - o *develop*: The primary integration branch for ongoing development.
 - o *feature/<feature-name>*: All new work must be done in a feature branch, created from *develop*.
 - o Pull Requests (PRs) are mandatory for merging feature branches into development. Each PR must be reviewed and approved by at least one other team member.

4.2. Document Management

- **Primary Platform (Formal Documents):** Microsoft OneDrive will serve as the central repository for all formal, text-based project documentation, including this project plan, user guides, meeting minutes and research materials.
- **Collaborative Editing:** Microsoft Office 365 (Word, Excel, PowerPoint) will be used for real-time collaborative editing and commenting on these documents.
- **Secondary Platform (General Assets):** Google Drive will be used for storing other project-related assets, including video files and other ad-hoc materials, to ensure easy access for all team members.
- **Backup Strategy:** In addition to cloud storage on both platforms, critical documents will be backed up manually on a weekly basis to a separate offline storage device.

4.3. Design and Architectural Asset Management

To ensure clarity and consistency in design and planning, the following tools and processes will be used:

- **Diagramming Tool (Visual):** *draw.io (Diagrams.net)* will be the standard tool for creating complex visual assets such as system architecture diagrams, flowcharts and database schemas. All source *.drawio* files will be stored in a dedicated 'Diagrams' folder within the team's Google Drive repository to leverage its direct save integration.
- **Diagramming Tool (Text-based):** *Mermaid.js* will be used for creating simpler diagrams directly within Markdown files. This approach is ideal for developer-focused documentation where diagrams need to be version-controlled alongside code, such as architectural decision records in the project wiki or sequence diagrams within GitHub Issues and Pull Requests.
- **UI/UX Prototyping Tool:** *Figma* will be used for all high-fidelity wireframes and interactive user interface prototypes. A central Figma project will be maintained and links to specific frames will be embedded directly into the corresponding GitHub Issues to ensure a clear and direct link between design specifications and development tasks.

5. Risk Management

A proactive approach to risk management is essential. The following register identifies potential risks and outlines a plan to address them. This register will be reviewed and updated at the beginning of each sprint.

ID	Risk description	Impact	Likelihood	Risk owner	Mitigation and contingency plan
R1	Technical skill gap: A team member may lack the necessary expertise in a required technology	Medium	Medium	Project lead	Mitigation: Promote a culture of knowledge sharing and pair programming. Allocate dedicated time within sprints for learning and experimentation. Encourage the use of online tutorials and documentation. Contingency: If a critical gap persists, the team will consult with the project supervisor for expert guidance or an alternative approach.
R2	Team member unavailability: A key member becomes unavailable for an extended period due to illness or personal emergency.	High	Low	Project lead	Mitigation: Enforce a strict policy of daily code commits to GitHub and maintain high-quality documentation. Implement a "buddy system" where knowledge for critical components (backend, AI core) is shared between two members. Contingency: The "buddy" will take over the primary responsibilities. The project lead will re-allocate tasks across the

					remaining team members and adjust the sprint scope if necessary.
R3	Scope creep: The client or team introduces new features or changes that are outside the original project scope.	Medium	Medium	Project lead	Mitigation: All change requests must be formally documented as a GitHub Issue. The team will perform an impact analysis to assess the effect on the timeline and resources. Contingency: The project lead will present the impact analysis to the project supervisor to get formal approval before any changes are incorporated into the project backlog. Low-impact changes may be deferred to a "post-prototype" backlog.
R4	Integration issues: The frontend, backend and AI modules fail to communicate correctly, causing significant delays late in the project.	High	Medium	All team members	Mitigation: Define clear API contracts between services early in the project. Conduct frequent integration testing throughout the development cycle. Implement a shared "mock server" so frontend and backend can develop in parallel. Contingency: Allocate a dedicated "integration week" before the final submission to resolve any complex integration bugs.
R5	AI framework underperformance: The LLM agent	High	Medium	AI engineers	Mitigation: Employ a Retrieval-Augmented Generation (RAG) pattern to ground the LLM's

	hallucinates, provides inaccurate recommendations.				responses in a trusted knowledge base. Rigorously validate the model's performance. Contingency: If the LLM is unreliable, implement a stricter rule-based NLP system as a fallback for critical recommendations.
R6	Simulated data unrealistic: The patient data simulator produces data that is too simplistic or does not accurately reflect real-world health patterns.	Medium	Medium	Backend developer, AI engineer	Mitigation: Research common patterns in chronic disease data. Incorporate randomness and variability into the simulator to mimic real patient behaviour. Contingency: Consult with the project supervisor or external resources to refine the simulation logic. Use a public, anonymised health dataset as a reference to validate the simulator's output distribution.

Table 8: Risk register

Part 2: Project Overview and Plan

This section details the project itself, covering the problem it aims to solve, its boundaries, objectives, technical architecture and the roadmap for its execution.

6. Project Background and Problem Statement

6.1. Background

The management of chronic diseases like Type 2 Diabetes is a persistent challenge in modern healthcare. Patients are often overwhelmed by the need to constantly monitor various health metrics (blood glucose, diet, activity, etc.) and struggle to interpret this data effectively. Healthcare providers, in turn, lack the tools for real-time monitoring and are often limited to providing guidance during infrequent appointments, relying on incomplete, patient-recalled information. This results in a reactive, rather than proactive, approach to care.

6.2. Problem Statement

Chronic disease patients lack a unified platform that integrates their fragmented health data and provides real-time, actionable and personalised guidance. Existing digital health solutions often act as simple data repositories and do not fully leverage the power of artificial intelligence to detect early warning signs, predict adverse events or provide context-aware recommendations. BioTective seeks to address this gap by developing a prototype solution that serves as an intelligent companion for patients and a powerful monitoring tool for clinicians, ultimately aiming to improve health outcomes through timely, AI-driven interventions.

7. Project Goals and SMART Objectives

7.1. Project Goal

To successfully design, develop, test and deliver a high-fidelity prototype of an AI-Enabled Digital Health Platform that meets all the functional and non-functional requirements outlined in the project proposal by the end of the semester.

7.2. SMART Objectives

Specific

Develop a cross-platform application (mobile and web) with distinct patient and clinician dashboards, an AI-powered recommendation engine, an automation layer for alerts and a patient-facing chatbot.

Measurable

Successfully complete all user stories defined in the product backlog, achieve a code review approval rate of 100% for all merged PRs and ensure the final prototype passes all end-to-end testing scenarios without critical bugs.

Achievable

The project scope is defined by six clear milestones and will be developed by a skilled five-person team using a modern, efficient technology stack. The use of a BaaS platform and high-level frameworks make the timeline feasible.

Relevant

The prototype directly addresses the problem statement provided by BioTective, demonstrating a solution that can enhance patient outcomes and improve the efficiency of chronic disease care, aligning with the company's mission.

Time-bound

The entire project, from planning to final prototype demonstration and documentation, will be completed within the official timeline of the COS40005 Computing Technology Project A unit.

8. Scope, Assumptions and Constraints

8.1. In-Scope

- Development of all features outlined in the six project milestones.
- A functional prototype integrating patient and clinician views.
- An AI-powered recommendation engine and an automation layer (LAM Triggers).
- A functional, AI-powered patient-facing chatbot for health data Q&A.
- Implementation of robust, database-level role-based access control for security.
- Development and use of a custom patient data simulator for all development and testing.

8.2. Out-of-Scope

- Deployment to a live, production environment with real patient data.
- Integration with real, physical health monitoring devices (e.g., Bluetooth glucose meters).
- The legal and financial process of achieving HIPAA compliance and signing a Business Associate Agreement (BAA).
- User onboarding flows, billing/subscription features and company-level administrative dashboards.

8.3. Assumptions

- The team will have access to all necessary software and development tools (IDEs, Flutter SDK, Python, etc.).
- The project requirements as defined in the proposal will remain stable throughout the project lifecycle.
- The team has the necessary collective skills to execute the project plan.
- The project supervisor will be available for regular consultation and guidance.

8.4. Constraints

- The project must be completed within the fixed semester timeline.
- The prototype will be developed using only simulated data due to the ethical and legal constraints of handling Protected Health Information (PHI).
- The budget for using third-party APIs (e.g., LLM providers) is limited and must be managed carefully.

9. Stakeholders

Identifying and understanding the expectations of all stakeholders is critical to project success. The following table outlines the key individuals and groups involved in this project, their roles and their primary interests.

Stakeholder	Role	Interest/Expectations
BioTective Sdn Bhd	Client	Expects a functional, polished prototype that validates their product concept and can be used for future investment pitches or as a foundation for a production build.
Ts. Dr. Vong Wan Tze	Supervisor	Expects the team to follow a structured project management methodology, demonstrate technical proficiency and deliver a high-quality project that meets the client's requirements and technical standards. Provides guidance on the project's technical direction and execution.
Dr. Brian Loh Chung Siong	Unit convener	Expects the team to adhere to all academic requirements and Unit Learning Outcomes, meet all submission deadlines and conduct themselves professionally. Ensures the project satisfies academic requirements and standards.
Patients and clinicians	End users	(Represented by personas) Expect an intuitive, reliable and

		useful platform that simplifies health management and provides valuable, trustworthy insights.
Development team	Executors	Aims to successfully deliver the project, gain practical experience in building a complex AI application and achieve a high academic grade.

Table 9: Stakeholders and their interest/expectations

10. Detailed Solution Approach and System Architecture

The proposed architecture is a modern, decoupled system designed for scalability, security and rapid development, leveraging best-in-class technologies for each component. This design is the result of a rigorous analysis of the cross-platform, backend and AI framework landscapes, prioritising a cohesive and efficient technology stack capable of meeting all project requirements.

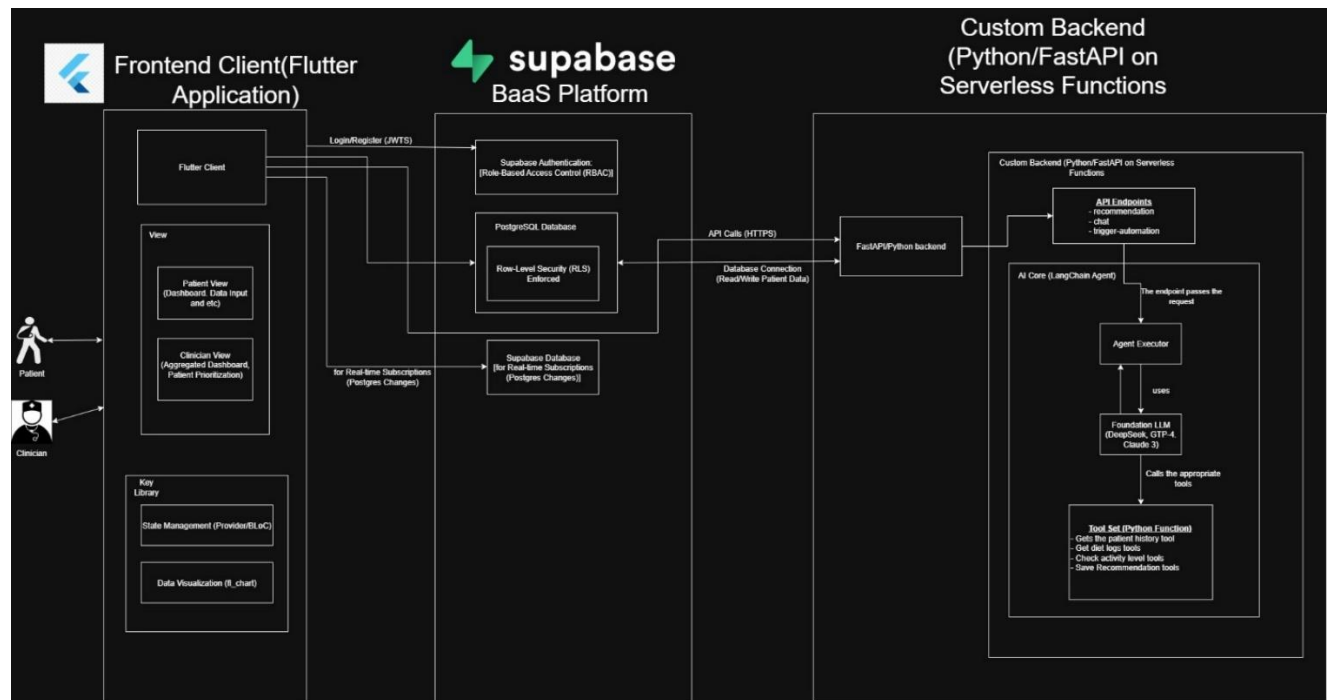


Figure 16: Conceptual diagram of multi-layer technical architecture

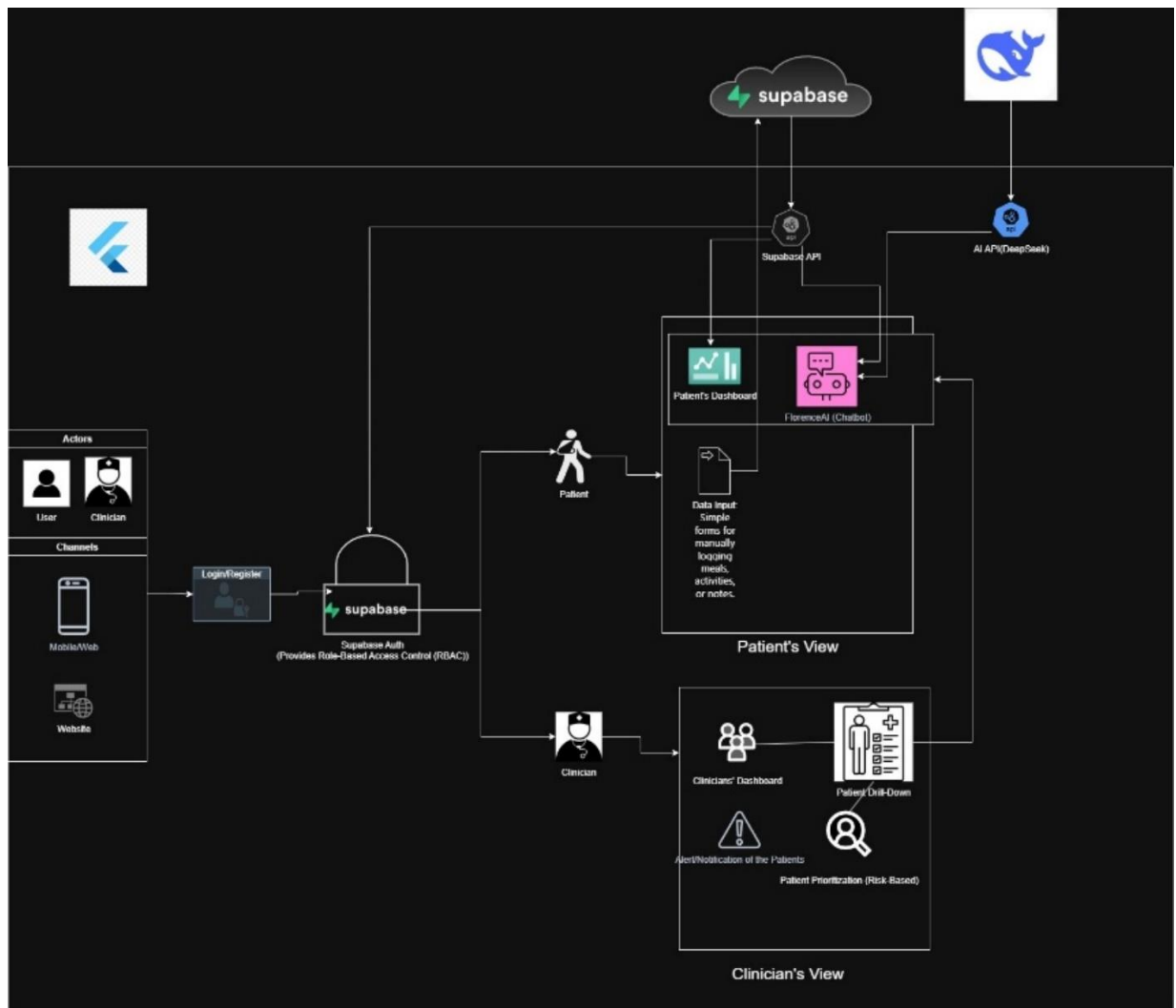


Figure 17: System architecture

10.1. Preliminary Design of Patient-facing Dashboard

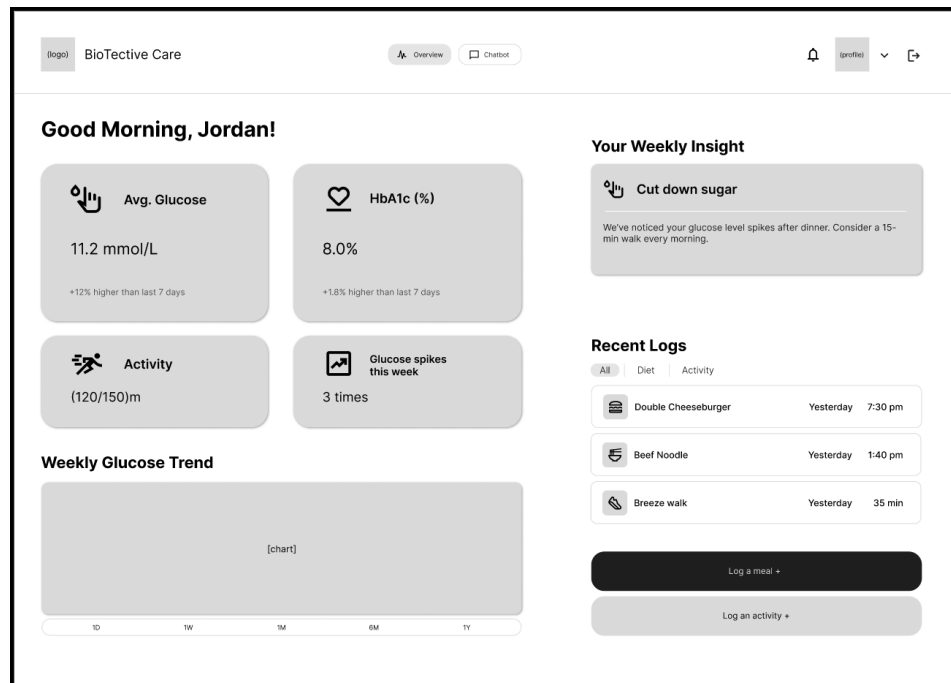


Figure 1: Main dashboard (website)

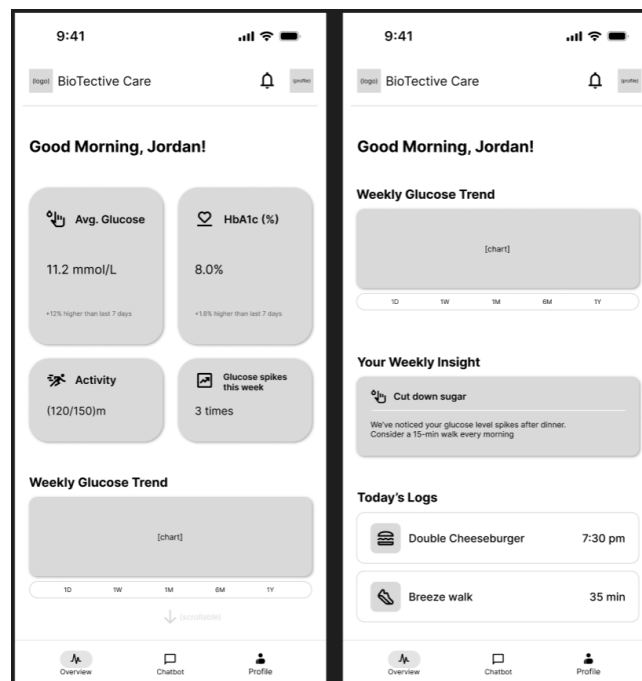


Figure 2: Main dashboard (mobile application)

Purpose

To provide an immediate, at-a-glance summary of the patient's current health status, recent trends, and the most critical AI-driven recommendation.

Upon navigating to the home screen, patients are greeted with a personalised welcome, such as "Good morning, [Patient Name]," fostering a sense of engagement. Displayed at the top are four key metric cards, offering vital real-time information. These include the latest glucose reading, complete with a timestamp and a color-coded indicator (e.g., green for in-range, yellow for elevated, red for high) to signify its status. Alongside this, a progress ring or bar visually represents today's activity, showing progress towards a daily goal (e.g., "1,500/5,000 steps" or "15/30 minutes"). The latest estimated HbA1c value is also presented, offering a crucial longer-term perspective on glucose control. A dedicated and visually distinct AI Insight Card captures immediate attention, displaying the most recent personalised recommendation generated by the LAM system. This card features a concise title, such as "A Tip for Your Evening Meals," followed by a short, empathetic summary of the insight. For example, it might state, "I've noticed your glucose levels tend to spike after dinner. Consider a short walk afterwards to help manage it.". The central feature of the home screen is an interactive line chart, powered by the `fl_chart` library, which visualises the patient's glucose readings over the last seven days. The X-axis denotes days of the week, while the Y-axis represents glucose levels (in mg/dL or mmol/L). This chart allows users to tap or hover over data points to view exact values and timestamps, with shaded areas clearly indicating the target glucose range.

The layout adapts responsively for mobile and web. On mobile devices, the interface presents a single, vertical scroll layout where Key Metric Cards are stacked, followed by the AI Insight Card and the main chart. For web users, a multi-column layout optimises screen space, potentially arranging Key Metric Cards horizontally at the top. The main chart might occupy the left side, while the AI Insight Card and a summary of recent activities or meals are presented in a right-hand sidebar. The "Quick-Log" function is integrated as a prominent button within the header for easy access.

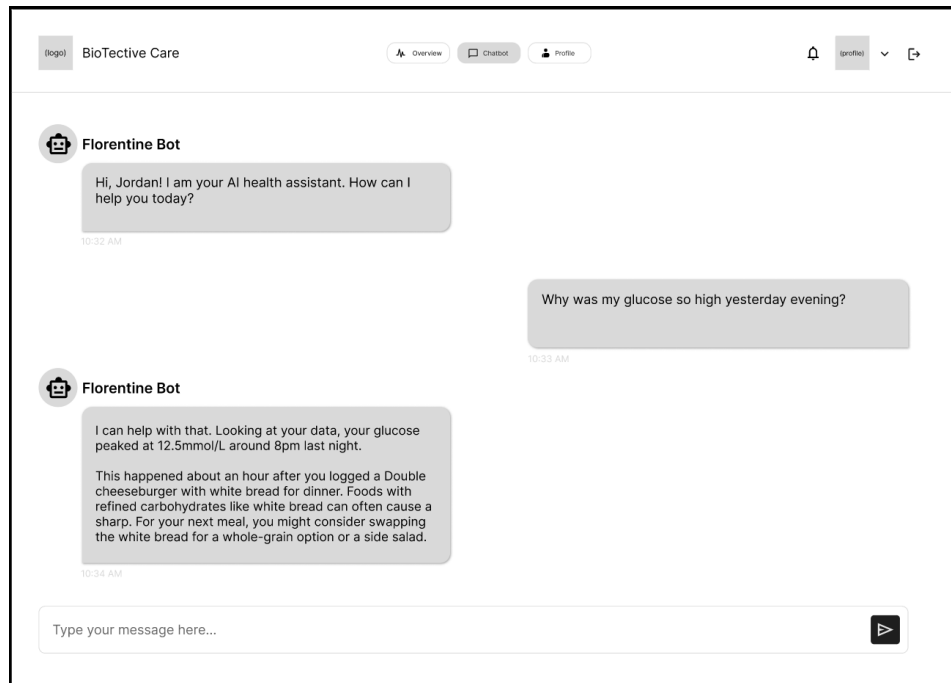


Figure 3: AI assistant chat interface (website)

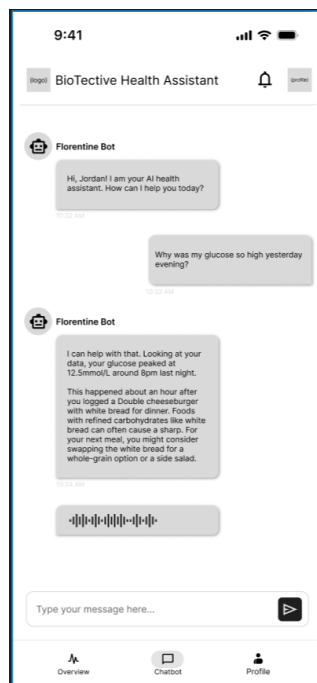


Figure 4: AI assistant chat interface (mobile application)

Purpose

To offer an interactive, conversational way for patients to ask questions about their health data and receive AI-generated, personalised responses.

The AI Chatbot screen provides a natural language interface for patients to interact with their health data. It features a familiar chat message interface, with user queries aligned to one side and the AI's insightful responses to the other, creating a clear dialogue flow. At the bottom of the screen, a text input field allows the user to type their questions, accompanied by a "Send" button. To enhance the user experience, a subtle "thinking" indicator animation appears while the LangChain agent is processing a request, providing visual feedback that the system is actively generating a response.

The core functionality of the chatbot remains consistent across both mobile and web platforms. On the web, users will benefit from typical keyboard shortcuts, such as pressing Enter to send a message. The chat interface might be presented within a modal window or a dedicated section of the main interface, rather than a full-screen takeover, allowing for greater multitasking flexibility.

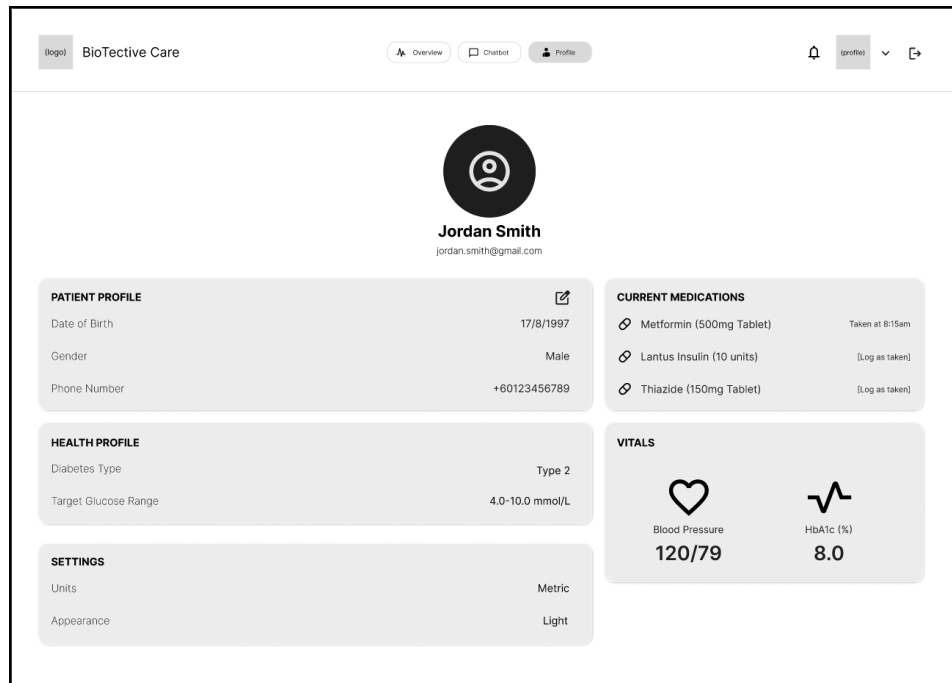


Figure 5: Patient's profile page (website)

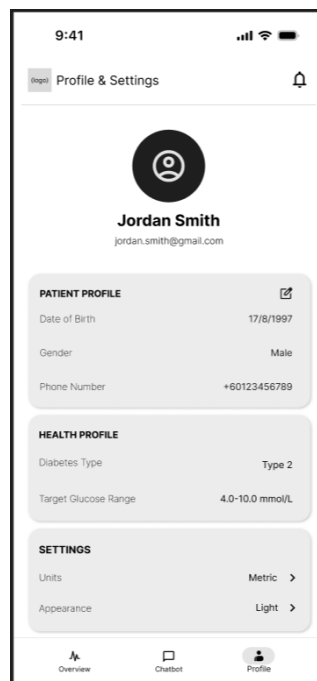


Figure 6: Patient's profile page (mobile application)

Purpose

To allow the user to manage their account information, customise app preferences, and control their data.

The Profile & Settings screen provides patients with comprehensive control over their account, application preferences, and data. Within the Account Information section, users can view and edit personal details such as their name and email, alongside an option to change their password securely. The Health Profile section displays non-editable key health information, including their Diabetes Type and the name of their linked Clinician, for quick reference. A dedicated Preferences section allows for customisation of the app's behaviour, featuring a segmented control to switch between mg/dL and mmol/L for glucose measurement units, and a toggle to select between Light and Dark mode themes. Crucially, a list of Notification Settings enables granular control over specific LAM Triggers, allowing patients to enable or disable alerts such as "Reminders for post-meal activity," "Motivational prompts for low-activity days," and "Weekly summary report," tailoring the app's proactiveness to their personal needs. Finally, a clearly marked "Logout" button is positioned at the bottom of the screen for account security.

This screen utilises a straightforward list or form-based layout, ensuring consistent usability across both mobile and web platforms with minimal design adjustments. On the web, it may be integrated as part of a tabbed settings interface within a larger user portal.

10.2. Preliminary Design of Clinician-facing Dashboard

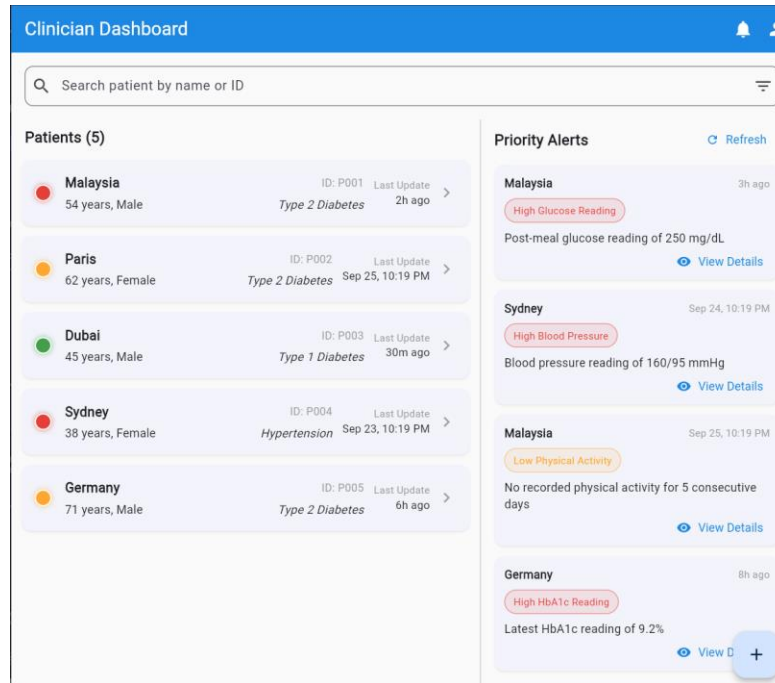


Figure 7: Main dashboard

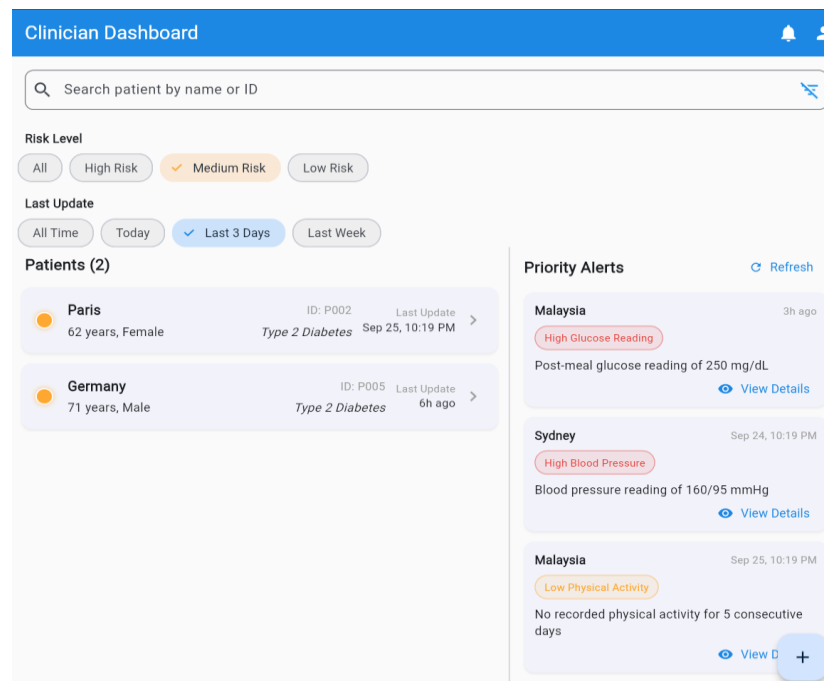


Figure 8: Sort function

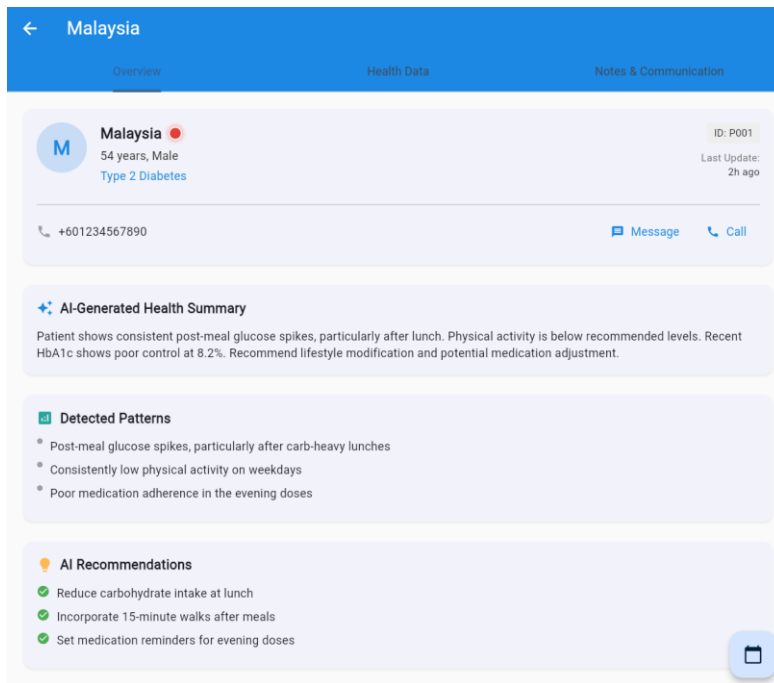


Figure 9: Patient's information overview

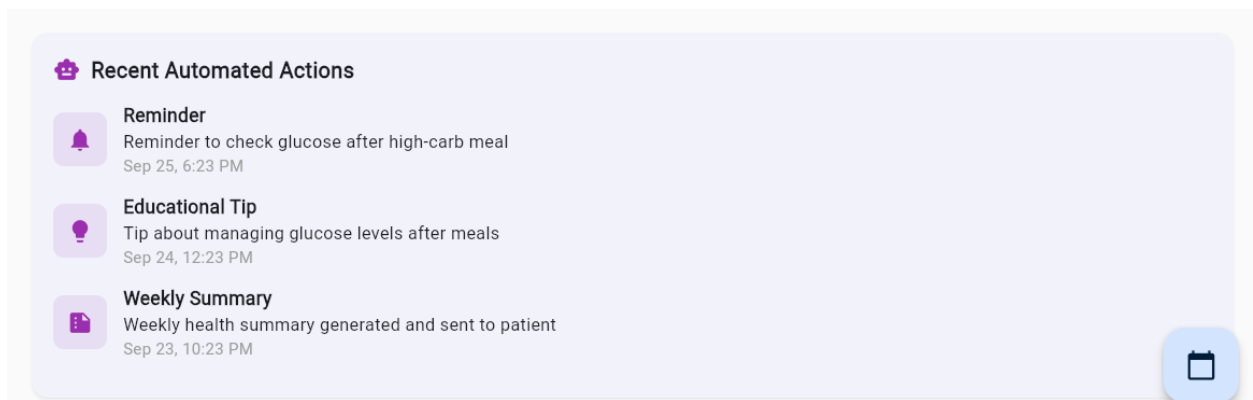


Figure 10: Recent automated actions

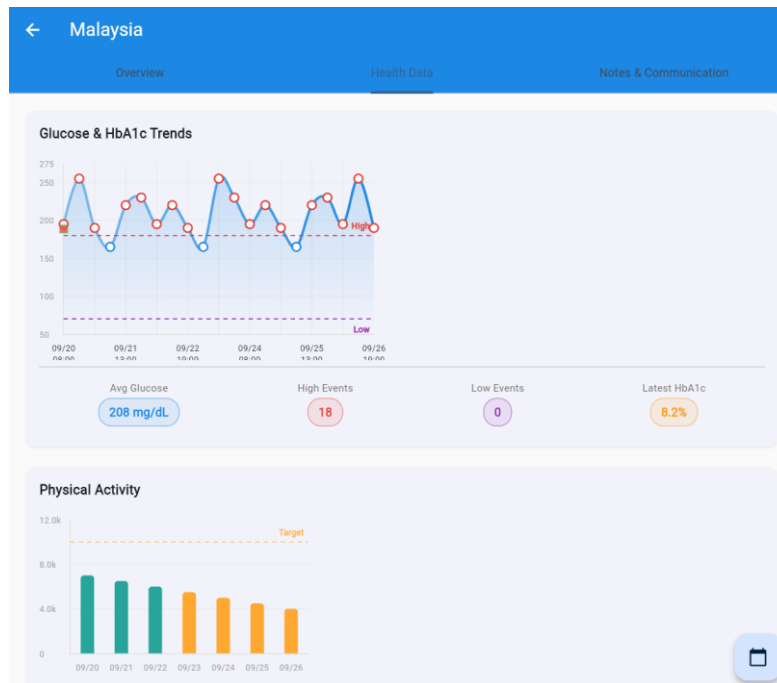


Figure 11: Glucose and HbA1c trends, and physical activity

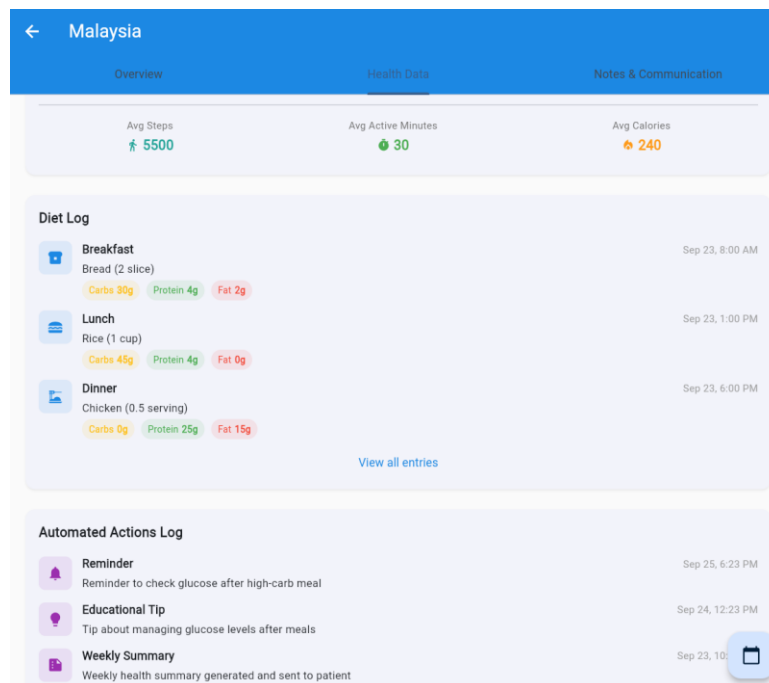


Figure 12: Diet log and automated actions log

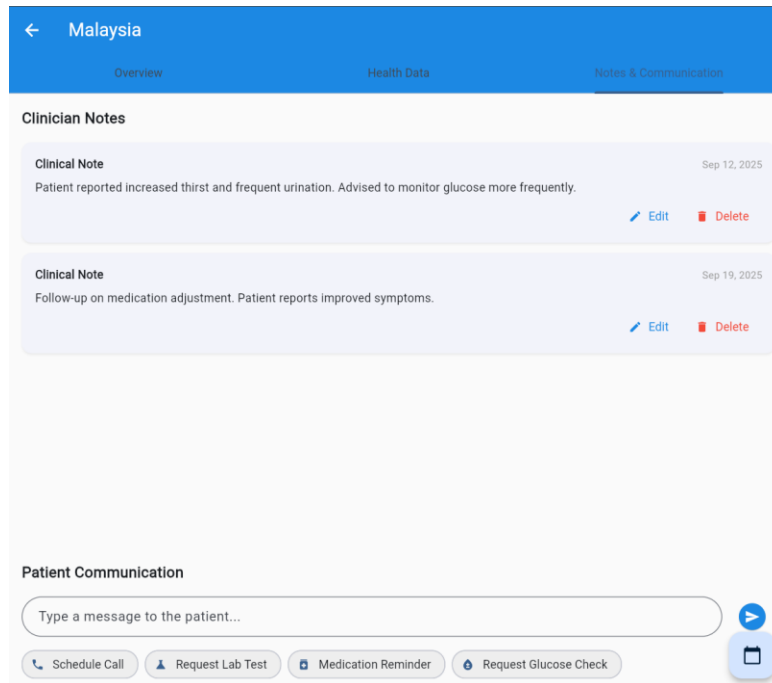


Figure 13: Notes and communication

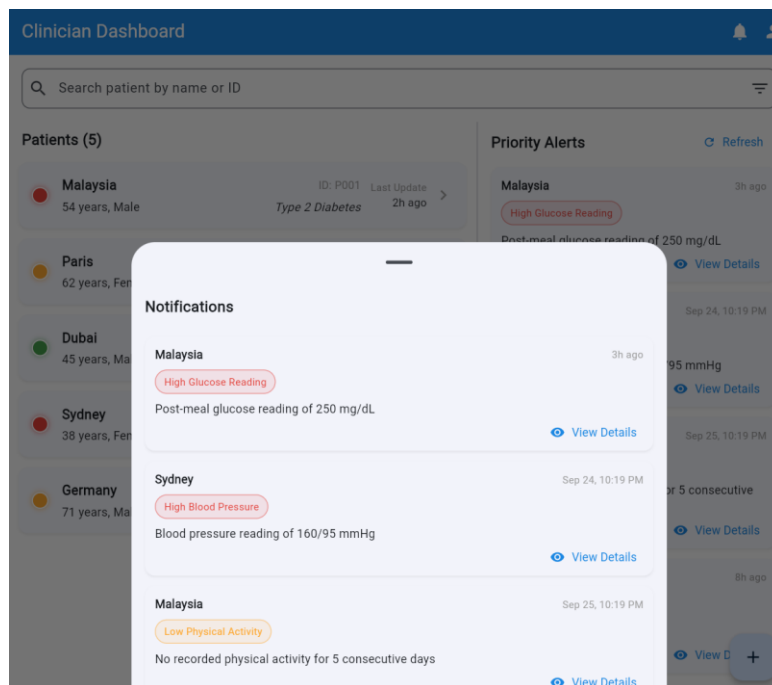


Figure 14: Notification centre

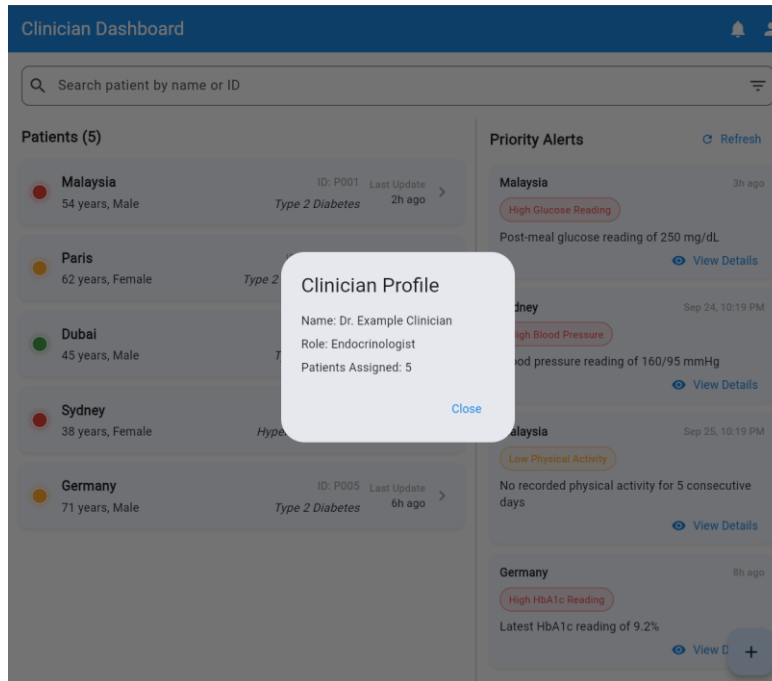


Figure 15: Clinician profile

The wireframe draft depicts a clinician-facing dashboard for remote patient monitoring, designed to support fast triage, at-a-glance risk assessment, and smooth drill-down into individual patient details. The home screen presents a patient roster with concise metrics, coloured risk indicators, and a recent alerts stream, complemented by quick filters to surface high-priority cases. Selecting a patient opens a detail view featuring trend charts, time-aligned alert history, and a clinician notes area to document assessments and interventions. The layout emphasises readability, scannability, and minimal friction, such as clear hierarchy, consistent spacing, touch-sized targets, and colour used primarily for status/risk cues. States for loading, empty, and error are accounted for, with mock data powering the draft to validate layout and interactions while preserving a clean path to real integrations. Accessibility and responsiveness guide the design for tablet and desktop, showing more patients and wider charts on larger screens. Assumptions include precomputed risk scoring and alert metadata sufficient for prioritisation, with planned enhancements such as chart time-range controls, inline alert actions, search, exportable summaries, and role-based views.

10.3. Frontend Architecture: A Cross-Platform Strategy

Rationale

Flutter was chosen as the definitive solution to meet the project's core "code once, deploy anywhere" requirement. Its architecture of compiling to native code and rendering its own UI via the high-performance Skia graphics engine provides significant advantages. This approach guarantees a fluid, responsive user experience across mobile and web platforms while giving the team complete control over the UI for a polished, bespoke application.

Alternatives Considered

React Native was evaluated but ultimately dismissed. While a premier choice for mobile, extending it to web and desktop requires integrating separate, community-maintained libraries, forcing the development team to act as system integrators and manage a fragmented build process. *Quasar Framework (Vue.js)* was also a strong contender due to its exceptional development velocity and best-in-class web output. However, Flutter was ultimately selected because its superior performance ceiling and UI fidelity were deemed more critical for a premium, user-centric health application.

State Management

The team will adopt a standardised state management solution, likely *Provider* or *BLoC*, to manage application state predictably and prevent UI inconsistencies.

Data Visualisation

The *fl_chart* library has been selected for its rich feature set, extensive customisation and strong community support, making it perfect for rendering the required trend charts and weekly summaries.

10.4. Backend Architecture: An AI-Centric Python Core

Rationale

This choice was non-negotiable, driven by Python's unrivalled ecosystem of mature libraries for artificial intelligence, machine learning and data science, which are central to the project's success. FastAPI was selected as the web framework for its high performance, asynchronous capabilities and automatic generation of interactive API documentation, which significantly accelerates development and integration testing.

Alternatives Considered

While other backend languages like Node.js or Go offer high performance, they lack the extensive and mature AI/ML libraries available in Python. Adopting them would necessitate creating complex inter-service communication with a separate Python service for AI tasks, introducing unnecessary architectural complexity. A monolithic Python backend is the most direct and efficient approach.

Deployment

The Python backend will be deployed as serverless functions (e.g., on Google Cloud Functions or Vercel), managed via the chosen BaaS platform to minimise DevOps overhead.

10.5. Data and BaaS Architecture: A Secure, Open-Source Foundation

Rationale

Supabase was chosen over its main competitor, Firebase, for its open-source nature and its foundation on a standard, powerful *PostgreSQL* database. This provides the project with the full power of a relational database, including complex queries and transactions, while benefiting from the BaaS features that accelerate development, such as integrated authentication and auto-generated APIs.

Security Cornerstone

A critical factor in this decision was Supabase's robust implementation of PostgreSQL's *Row-Level Security (RLS)*. RLS allows for the creation of fine-grained security policies directly in the database, ensuring that data segregation between patients and clinicians is enforced at the lowest, most secure level possible. This is a non-negotiable requirement for an application handling sensitive health information.

10.6. AI Core Architecture: An Agentic System

Rationale

The project's AI core will be implemented as a single, autonomous agent that can reason and perform actions. This "Large Action Model" (LAM) system is best constructed using a framework designed for this purpose. LangChain was selected due to its mature and powerful "Agent Executors" and the industry's largest ecosystem of pre-built tools and integrations. This makes it the most capable and extensible choice for an application where the primary AI function is to execute tasks (e.g., fetch data, analyse patterns, trigger alerts).

Alternatives Considered

LlamaIndex was a strong alternative, noted for its simpler API and best-in-class capabilities for Retrieval-Augmented Generation (RAG). However, since the project's immediate priority is action-oriented tool use rather than knowledge retrieval, LangChain's more mature agentic execution capabilities were a better fit. Multi-agent frameworks like CrewAI were also reviewed but deemed overly complex and unsuitable for a single-agent system.

Model

A powerful foundation Large Language Model (e.g., GPT-4, Claude 3) with strong tool-calling capabilities.

Function

The agent acts as the system's "brain." It receives context (e.g., patient data) and uses its reasoning ability to decide on a course of action. It can then use a predefined set of "tools" (custom Python functions) to execute this plan. For instance, it could use a tool to fetch the patient's full diet history from Supabase, another to check for recent activity and then synthesise all this information into a comprehensive, empathetic recommendation. This is the core of the "Large Action Model" (LAM) system.

11. Detailed Product Backlog

This product backlog serves as the definitive list of project requirements, broken down into actionable user stories in alignment with agile development methodology. The backlog includes a comprehensive mix of functional requirements (user-facing features), non-functional requirements (such as security and performance) and technical/system requirements (such as project setup and testing). Each story has been assigned a priority, estimated in story points and allocated to a specific sprint to form the development roadmap. This structure provides a clear and transparent plan for the iterative delivery of the project.

ID	User story	Priority	Story points	Acceptance criteria	Sprint
EPIC 1: Project foundation and patient view (M1 and M2)					
T1.1	As a developer, I want to set up the project structure with a GitHub repo and CI/CD pipeline so that we have a foundation for collaborative work.	High	3	Repo created, GitFlow branches configured, basic linting action runs on PR.	1
T1.2	As a developer, I want to set up the Supabase project with a complete database schema so that we can store user and health data.	High	5	All tables (users, patients, clinicians, glucose, diet, activity) are created with correct relationships and constraints.	1

T1.3	As a patient/clinician, I want to securely sign up, log in and log out so that I can access the platform and my data is protected.	High	8	Users can register. Users can log in with email/password. JWT is issued on login. RLS policies for basic access are in place.	1
T1.4	As a developer, I need a Python script to simulate and populate the database with realistic patient data so that we can develop and test features.	High	8	Script generates data for multiple patients over several weeks. Data includes realistic patterns like meal-related glucose spikes.	1
T2.1	As a patient, I want to see a dashboard with visualisations of my key health metrics (glucose, HbA1c, diet, activity) so that I can understand my trends.	High	13	Dashboard UI is built in Flutter. <i>fl_chart</i> widgets display data from Supabase. UI is responsive for mobile/desktop.	1
EPIC 2: AI core and automation (M3 and M4)					
T3.1	As an AI Engineer, I want to build a LangChain agent that can reason about patient data and use tools to fetch additional information from the database.	High	21	Agent is initialised with a base LLM. At least two tools are created (e.g., <i>get_recent_glucose</i> , <i>get_diet_history</i>). Agent can successfully call tools and use their output.	2
T3.2	As a patient, I want to receive personalised	High	13	The LangChain agent synthesises data and	2

	recommendations (e.g., meal substitutions, activity suggestions) based on my data patterns.			predictions to generate a text recommendation. Recommendation is displayed on the patient dashboard.	
T4.1	As a developer, I want to create an automation layer (LAM Triggers) that runs periodically to check for abnormal health patterns in the database.	High	8	A scheduled serverless function is created that queries the database for trigger conditions (e.g., high glucose, low activity).	2
T4.2	As a patient, I want to receive a timely reminder (e.g., push notification) to take a short walk after a high-carb meal is detected.	Medium	5	A LAM Trigger identifies a high-carb meal and a subsequent glucose spike, triggering a notification event.	2
EPIC 3: Clinician view and finalisation (M5 and M6)					
T5.1	As a clinician, I want to see a dashboard with a list of my assigned patients, highlighting those who need urgent attention.	High	13	Clinician dashboard UI is built. RLS policies ensure clinicians only see their assigned patients. A risk-scoring system flags high-risk patients.	3
T5.2	As a patient, I want to use a chatbot to ask simple questions about my health data (e.g., "What was my	Medium	13	Chat UI is built in Flutter. The LangChain agent is connected to the chat interface to provide AI-	3

	average glucose this week?").			generated responses based on the patient's data.	
T6.1	As a developer, I want to conduct end-to-end system testing using a multi-patient dataset to ensure all components function correctly together.	High	8	A comprehensive test plan is executed. All critical bugs are logged and resolved.	3
T6.2	As a team, we want to finalise all project documentation, including a comprehensive user guide and technical documentation, for final submission.	High	13	User guide for patients/clinicians is written. Technical documentation covers architecture and setup instructions.	3

Table 10: Backlog register

12. Quality Assurance Plan

The team is committed to delivering a high-quality prototype. Quality is not just a final check but a continuous process integrated throughout the development lifecycle.

12.1. Code Quality Standards

Style Guides

The team will strictly enforce official style guides: *PEP 8* for Python and *Effective Dart* for Flutter.

Linters and Formatters

Automated tools will be integrated into the development environment and CI pipeline. *Black* and *Pylint* for Python, and Dart's built-in *dart format* and linter for Flutter. Commits that do not pass linting will be blocked.

12.2. Testing Strategy

Unit Testing

Developers are responsible for writing unit tests for their code, especially for critical backend logic, utility functions and AI model inputs/outputs. The target is to achieve a reasonable level of code coverage for business-critical modules. Tools: *pytest* for Python, *flutter_test* for Flutter.

Integration Testing

At the end of each week, the team will conduct integration tests to ensure that the frontend can successfully communicate with the backend API and that the API can correctly interact with the Supabase database.

End-to-End (E2E) System Testing

During the final sprint, the team will perform comprehensive E2E testing based on the user stories. This involves testing the entire workflow from the user's perspective (e.g., logging a meal, seeing a glucose spike, receiving a recommendation).

User Acceptance Testing (UAT)

In the final two weeks, team members will engage in peer-UAT, where they test the features developed by others, acting as both a patient and a clinician to identify usability issues and bugs.

12.3. Defect Management

Reporting

All bugs and defects must be reported as a *GitHub Issue* using a standardised bug report template.

Tracking

Each bug will be labelled, assigned a priority (Critical, High, Medium, Low) and assigned to a team member.

Resolution

A bug is considered resolved only after a fix has been committed, merged and the fix has been verified by the original reporter or another team member.